

Learn Low-Level Design in 25 Days: Day 21 — Designing a Chat System (WhatsApp/Slack Style)

The Everyday Miracle of Chat

TechPreneur

Published November 2, 2025

Free: No

Read on Freedium · original

Contents

The Core Idea — How Chats Really Work 0

Key Entities 0

Step 1: Modeling the Core Classes 0

Step 2: The Flow of a Message 0

The MessageStore ensures persistence: 0

Visual Overview 0

Step 3: Adding Real-Time and Offline Support 0

Step 4: Handling Group Chats 0

Step 5: Persistence and Scalability Considerations 0

Try It Yourself 0

Key Takeaways 0

What's Next 0

Ever sent a message on WhatsApp and got that double tick almost instantly? Or watched as your teammate on Slack typed "...is typing" before a reply popped up milliseconds later? Behind those seemingly simple interactions is a finely tuned low-level design — handling real-time delivery, message queues, persistence, and synchronization across devices.

Welcome to **Day 21** of Learn Low-Level Design in 25 Days. Today, we're going to **design a chat system** — the kind that powers Slack, WhatsApp, or Microsoft Teams. By the end, you'll understand how to structure classes, manage message flow, and design for **real-time scalability** while keeping the code clean and extensible.



This one's a milestone — because it's where communication meets concurrency, persistence, and clean design.

The Core Idea — How Chats Really Work

At its heart, a chat system is about **sending and receiving messages between users**. But that simple sentence hides a complex system of concerns:

- **Message delivery** — reliable and ordered (no missing or duplicated messages)
- **Real-time updates** — users should see new messages instantly
- **Offline support** — messages must sync when users reconnect
- **Persistence** — every message is stored and retrievable later
- **Scalability** — millions of concurrent users

Let's first break the chat world into simple components.

Key Entities

Entity	Description
User	Represents a person in the system
Chat	A conversation between one or more users (1-1 or group)
Message	The actual text/media sent by a user
MessageStore	Handles storage, retrieval, and ordering of messages
ChatService	Orchestrates sending, receiving, and notifying users
NotificationService	Sends delivery/read updates

Step 1: Modeling the Core Classes

Here's a simple object model for our chat system:

```
class User {
    String id;
    String name;
    List<Chat> chats;
}

class Chat {
    String id;
    List<User> participants;
    List<Message> messages;

    void addMessage(Message message) {
        messages.add(message);
    }
}

class Message {
    String id;
    User sender;
    String content;
    LocalDateTime timestamp;
    MessageStatus status;
}

enum MessageStatus {
    SENT, DELIVERED, READ
}
```

This defines the **domain layer** — clean, expressive, and decoupled from delivery or storage details.

Step 2: The Flow of a Message

Let's simulate what happens when Alice sends a message to Bob.

```
Alice → ChatService → MessageStore → NotificationService → Bob
```

Here's how we can model this in code:

```
class ChatService {
    MessageStore messageStore;
    NotificationService notificationService;

    void sendMessage(Chat chat, Message message) {
        messageStore.save(chat, message);
        notificationService.notifyUsers(chat, message);
    }
}
```

The `MessageStore` ensures persistence:

```
class MessageStore {
    Map<String, List<Message>> chatMessages = new HashMap<>();

    void save(Chat chat, Message message) {
        chatMessages.computeIfAbsent(chat.id, k -> new ArrayList<>()).add(message);
    }

    List<Message> getMessages(String chatId) {
        return chatMessages.getOrDefault(chatId, new ArrayList<>());
    }
}
```

And the `NotificationService` handles real-time delivery:

```
class NotificationService {
    void notifyUsers(Chat chat, Message message) {
        for (User user : chat.participants) {
            if (!user.equals(message.sender)) {
                System.out.println("Notifying " + user.name + ": New message from " +
message.sender.name);
            }
        }
    }
}
```

Visual Overview

```
+-----------+         +--------------+         +-----------------+
|   User A  | -----> |  ChatService | -----> |  MessageStore   |
+-----------+         +--------------+         +-----------------+
       |                          |
       |                          v
       |                +--------------------+
       +--------------> | NotificationService|
                        +--------------------+
```

Each layer has a **single responsibility** — the foundation of maintainable LLD.

Step 3: Adding Real-Time and Offline Support

In real-world systems like WhatsApp or Slack, delivery isn't just a function call — it's an event-driven system:

- **WebSockets** maintain a persistent connection for real-time delivery.
- **Message Queues** (Kafka, RabbitMQ) buffer and distribute messages reliably.

- **Delivery Receipts** update message states (`SENT → DELIVERED → READ`).

We can represent this in a **Publisher-Subscriber** fashion:

```
ChatService ---> MessageStore
            ---> NotificationService
                    ---> OnlineUserSessions
```

When a message is stored, an event is published; subscribers (like active user sessions) react to it asynchronously.

Step 4: Handling Group Chats

Group chats simply scale this pattern. The same message is delivered to multiple recipients — but delivery receipts become per-user.

```
class GroupChat extends Chat {
    Map<User, MessageStatus> messageReadStatus;
}
```

This structure helps track who has read which message — the "seen by 4" feature in WhatsApp or "read by @John" in Slack.

Step 5: Persistence and Scalability Considerations

In production, you'd replace in-memory data with scalable infrastructure:

Concern	Example Solution
Persistence	MongoDB, Cassandra, DynamoDB (for chat history)
Message Queue	Kafka / Redis Streams (for delivery pipeline)
Realtime Updates	WebSockets, Socket.io
Search	Elasticsearch for message search
Media Handling	CDN + S3 for attachments

Try It Yourself

1. **Add Typing Indicators** — Extend the `NotificationService` to broadcast when a user starts/stops typing.

2. **Implement Message Acknowledgment** — Design how a client might send a "delivered" or "read" status back to the server.
3. **Think about Offline Messages** — How would you queue messages for a user who is temporarily disconnected?

Key Takeaways

- A chat system is a mix of **real-time**, **persistent**, and **event-driven** design.
- Separate concerns — message storage, delivery, and notification should be independent.
- Use patterns like **Observer (Pub/Sub)** for notifying active users.
- Start simple, then scale — add WebSockets, queues, and distributed stores later.
- Focus on **clear domain modeling** before worrying about infrastructure.

What's Next

Tomorrow, we'll explore **"How to Write Class Diagrams for Interviews"** — a skill that turns your design ideas into clear, visual blueprints. You'll learn how to present your chat system design in a crisp UML diagram that interviewers love.

 Follow the full series here: Learn Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)

Save the list to stay on track — and keep building one real-world design at a time.